

1. ¿Qué es el modelado de datos?

El modelado de datos consiste en explorar estructuras orientadas a datos. Al igual que otros artefactos de modelado, los modelos de datos pueden utilizarse para diversos fines, desde modelos conceptuales de alto nivel hasta modelos de datos físicos. Desde la perspectiva de un desarrollador orientado a objetos, el modelado de datos es conceptualmente similar al modelado de clases. En el modelado de datos se identifican tipos de entidades, mientras que en el modelado de clases se identifican clases. Los atributos de datos se asignan a los tipos de entidades, del mismo modo que se asignan atributos y operaciones a las clases. Existen asociaciones entre entidades, similares a las asociaciones entre clases: relaciones, herencia, composición y agregación son conceptos aplicables al modelado de datos.

El modelado de datos tradicional se diferencia del modelado de clases en que se centra exclusivamente en los datos. Los modelos de clases permiten explorar tanto el comportamiento como los aspectos de datos del dominio, mientras que un modelo de datos solo permite explorar los problemas relacionados con los datos. Debido a este enfoque, los modeladores de datos suelen ser mucho mejores a la hora de representar los datos correctamente que los modeladores de objetos. Sin embargo, algunas personas modelan métodos de base de datos (procedimientos almacenados, funciones almacenadas y disparadores) al modelar datos físicos. Depende de la situación, por supuesto, pero personalmente creo que es una buena idea y promuevo este concepto en mi [perfil de modelado de datos UML](#) (más adelante hablaré de ello). Aunque este artículo se centra en el modelado de datos, existen a menudo alternativas a los artefactos orientados a datos (no olvidemos el principio [de Modelos Múltiples](#) del Modelado Ágil). Por ejemplo, en el modelado conceptual, [los diagramas ORM](#) no son la única opción: además de los LDM, es bastante común crear [diagramas de clases UML](#) e incluso [tarjetas de Colaborador de Responsabilidad de Clase \(CRC\)](#). De hecho, mi experiencia indica que las tarjetas CRC son superiores a los diagramas ORM porque facilitan la [participación activa de las partes interesadas](#) en la creación del modelo. En lugar de una sesión de dibujo tradicional dirigida por [un analista](#), se puede facilitar la participación de las partes interesadas mediante la creación de [tarjetas CRC](#).

1.1 ¿Cómo se utilizan los modelos de datos en la práctica?

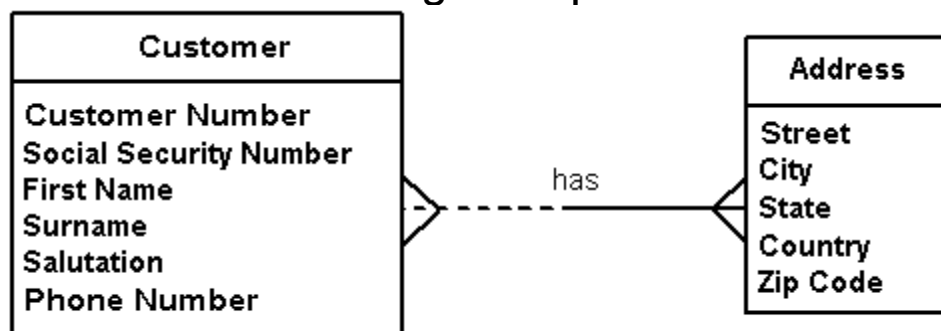
Aunque las cuestiones metodológicas se abordan [más adelante](#) , necesitamos analizar cómo se pueden utilizar los modelos de datos en la práctica para comprenderlos mejor. Es probable que encuentre tres estilos básicos de modelos de datos:

- **Modelos de datos conceptuales** . Estos modelos, a veces llamados modelos de dominio, se utilizan normalmente para explorar conceptos de dominio con las partes interesadas. En los equipos ágiles, los modelos conceptuales de alto nivel suelen crearse como parte de la fase [inicial de definición de requisitos](#), ya que se utilizan para explorar las estructuras y conceptos empresariales estáticos de alto nivel. En los equipos tradicionales, los modelos de datos conceptuales suelen crearse como precursores de los modelos de datos de dominio (LDM) o como alternativas a estos.
- **Modelos de datos lógicos (MDL)** . Los MDL se utilizan para explorar los conceptos del dominio y sus relaciones dentro del ámbito del problema. Esto puede aplicarse a una iniciativa específica o a toda la empresa. Los MDL representan los tipos de entidades lógicas, generalmente denominados simplemente tipos de entidad, los atributos de datos que describen dichas entidades y las relaciones entre ellas. Los MDL rara vez se utilizan en [iniciativas ágiles](#) , aunque sí son frecuentes en equipos tradicionales (donde, en la práctica, no suelen aportar mucho valor).
- **Modelos de datos físicos (MDP)** . Los MDP se utilizan para diseñar el esquema interno de una base de datos, representando las tablas de datos, las columnas de datos de dichas tablas y las relaciones entre ellas. Los MDP suelen ser útiles tanto en equipos ágiles como tradicionales, por lo que este artículo se centra en el modelado físico.

Aunque los diagramas LDM y PDM suenan muy similares, y de hecho lo son, el nivel de detalle que modelan puede ser significativamente diferente. Esto se debe a que los objetivos de cada diagrama son distintos: se puede usar un LDM para explorar conceptos de dominio

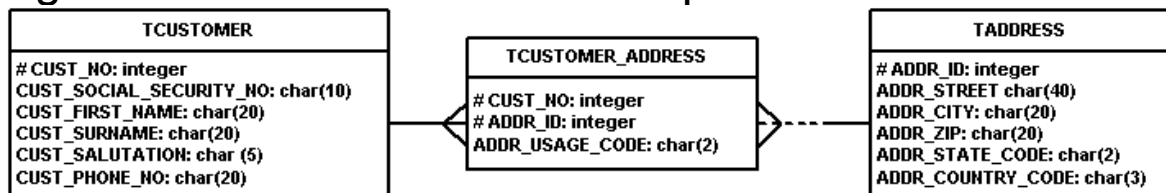
con las partes interesadas y un PDM para definir el diseño de la base de datos. La Figura 1 presenta un LDM simple y la Figura 2 un PDM simple, ambos modelando el concepto de clientes y direcciones, así como la relación entre ellos. Ambos diagramas aplican la [notación de Barker](#), resumida a continuación. Observe cómo el PDM muestra mayor detalle, incluyendo una tabla asociativa necesaria para implementar la asociación, así como las claves necesarias para mantener las relaciones. Más adelante se abordarán estos conceptos con mayor profundidad. Los PDM también deben reflejar los estándares de nomenclatura de bases de datos de la organización; en este caso, se añade una abreviatura del nombre de la entidad a cada nombre de columna y se introdujo de forma consistente una abreviatura para "Número". Un PDM también debe indicar los tipos de datos de las columnas, como entero y char(5). Aunque la Figura 2 no las muestra, las tablas de búsqueda (también llamadas tablas de referencia o tablas de descripción) sobre cómo se utiliza la dirección, así como los estados y países, están implícitas en los atributos *ADDR_USAGE_CODE*, *STATE_CODE* y *COUNTRY_CODE*.

Figura 1. Un modelo de datos lógico simple.



Copyright 2002-2006 Scott W. Ambler

Figura 2. Un modelo de datos físico simple.



Copyright 2002-2006 Scott W. Ambler

Una observación importante sobre las Figuras 1 y 2 es que no sigo al pie de la letra el método de Barker para nombrar las relaciones. Por

ejemplo, entre *Cliente* y *Dirección*, en realidad deberían existir dos nombres: «Cada CLIENTE puede estar ubicado en una o más DIRECCIONES» y «Cada DIRECCIÓN puede ser la sede de uno o más CLIENTES». Si bien estos nombres definen explícitamente la relación, personalmente creo que son ruido visual que recarga el diagrama. Prefiero nombres sencillos como «tiene» y confío en que mis lectores los interpreten en cada sentido. Solo añadiré más información donde sea necesario; en este caso, creo que no lo es. Sin embargo, una ventaja significativa de describir los nombres como sugiere Barker es que sirve como una buena prueba para comprobar si realmente se comprende la relación: si no se puede nombrar, probablemente no se entienda. Los modelos de datos pueden utilizarse eficazmente tanto a nivel empresarial como en los equipos de desarrollo. Los arquitectos empresariales suelen crear uno o más modelos de datos empresariales (LDM) de alto nivel que representan las estructuras de datos que sustentan la empresa; estos modelos se conocen comúnmente como modelos de datos empresariales o modelos de información empresarial. Un modelo de datos empresarial es una de las diversas vistas que [los arquitectos empresariales](#) de la organización pueden optar por mantener y respaldar. Otras vistas pueden explorar la infraestructura de red/hardware, la estructura organizativa, la infraestructura de software y los procesos de negocio (entre otros). Los modelos de datos empresariales proporcionan información que un equipo puede utilizar tanto como conjunto de restricciones como para obtener información valiosa sobre la estructura de su sistema. Los equipos de desarrollo suelen crear LDM como artefacto de análisis principal cuando su entorno de implementación es predominantemente procedimental, por ejemplo, cuando utilizan COBOL estructurado como lenguaje de implementación. Los LDM también son una buena opción cuando una iniciativa está orientada a los datos, como por ejemplo, cuando se está desarrollando un [almacén de datos](#) o un sistema de informes (aunque la experiencia parece indicar que los enfoques centrados en el uso funcionan incluso mejor). Sin embargo, los LDM suelen ser una mala elección cuando un equipo utiliza tecnologías orientadas a objetos o basadas en componentes, ya que los desarrolladores prefieren trabajar con diagramas UML, o cuando el equipo no está orientado a los datos. Como [aconseja el modelado ágil](#), [utilice el/los artefacto\(s\) adecuado\(s\)](#) para cada tarea. O, como probablemente le aconsejaría su abuelo, utilice la herramienta adecuada para cada tarea. Es importante tener en cuenta que los enfoques tradicionales de la [gestión de datos maestros \(MDM\)](#) suelen

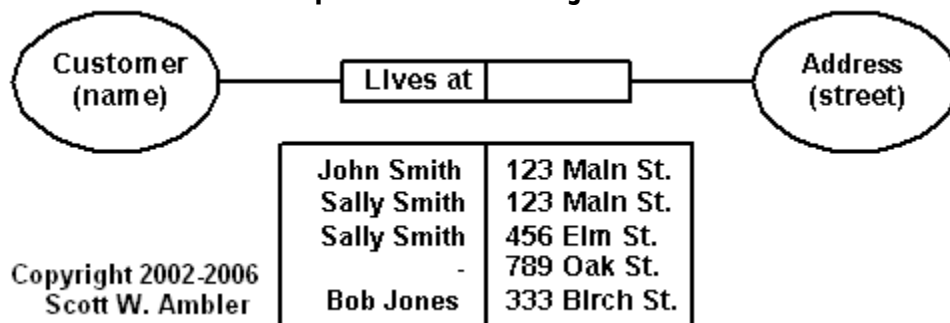
motivar la creación y el mantenimiento de LDM detallados, un esfuerzo que rara vez se justifica en la práctica si se considera el coste total de propiedad (TCO) al calcular el retorno de la inversión (ROI) de este tipo de iniciativas.

Cuando se utiliza una [base de datos relacional](#) para el almacenamiento de datos, se recomienda a los equipos crear un modelo de datos de producto (PDM) para modelar su esquema interno. En mi experiencia, un PDM suele ser uno de los artefactos de diseño fundamentales para [las iniciativas](#) de desarrollo de aplicaciones empresariales.

2.2. ¿Qué hay de...? ¿Modelos conceptuales?

[Halpin \(2001\)](#) señala que muchos profesionales de datos prefieren crear un [Modelo de Rol-Objeto \(ORM\)](#), como se muestra en [la Figura 3](#), en lugar de un Modelo de Datos Logísticos (LDM) para un modelo conceptual. La ventaja es que la notación es muy sencilla, algo que las partes interesadas pueden comprender rápidamente, aunque la desventaja es que los modelos crecen rápidamente. Los ORM permiten explorar primero ejemplos de datos reales en lugar de pasar directamente a una abstracción potencialmente incorrecta; por ejemplo, [la Figura 3](#) examina en detalle la relación entre clientes y direcciones. Para obtener más información sobre ORM, visite www.orm.net.

Figura 3. Un modelo simple de rol de objeto.



Mi experiencia es que la gente capturará la información en el mejor lugar que conozca. Por lo tanto, suelo descartar los ORM una vez que termino de usarlos. A veces uso ORM para explorar el dominio con las

partes interesadas, pero luego los reemplazo con un artefacto más tradicional como un LDM, un diagrama de clases o incluso un PDM. Como [especialista generalista](#) , alguien con una o más especialidades que también se esfuerza por adquirir habilidades y conocimientos generales, esta es una decisión fácil para mí; sé que esta información que acabo de "descartar" se capturará en otro artefacto (un modelo, las pruebas o incluso el código) que entiendo. Un especialista que solo entiende un número limitado de artefactos y, por lo tanto, "delega" su trabajo a otros especialistas no tiene esta opción. No solo se ven tentados a conservar los artefactos que crean, sino también a invertir aún más tiempo en mejorarlos. Los especialistas generalistas son más propensos que los especialistas a [viajar ligeros](#) .

2.3. Modelado de datos común Notaciones

[La Figura 4](#) presenta un resumen de la sintaxis de cuatro notaciones comunes de modelado de datos: Ingeniería de la Información (IE), Barker, IDEF1X y el [Lenguaje Unificado de Modelado \(UML\)](#) . Este diagrama no pretende ser exhaustivo, sino que su objetivo es proporcionar una visión general básica. Además, por brevedad, no pude representar el enfoque detallado para la nomenclatura de relaciones que sugiere Barker. Si bien proporciono una breve descripción de cada notación en [la Tabla 1](#), recomiendo encarecidamente el artículo de David Hay, [«Una comparación de técnicas de modelado de datos»](#), ya que profundiza más en el tema que yo.

Figura 4. Comparación de la sintaxis de las notaciones comunes de modelado de datos.

Notation	Information Engineering	Barker Notation	IDEF1X	UML
Multiplicities:				
- Zero or one				
- One only				
- Zero or more				
- One or more				
- Specific range	N/A	N/A	N/A	
Attributes:				
Names	N/A	Attribute Name: Type	attribute-name: Type	attributeName: Type
Primary key/unique identifier	N/A	# Attribute Name		attributeName <<PK>> {order=#}
Foreign key	N/A	N/A	attribute-name (FK)	attributeName <<FK>> {to=tablename}
Associations:				
Labels				
Entity roles	N/A	N/A	N/A	
Subtyping				
Aggregation				
Composition				
Or Constraint		N/A	N/A	
Exclusive Or (XOR) Constraint			N/A	

Tabla 1. Discusión sobre notaciones comunes de modelado de datos.

Notación	Comentarios
ES DECIR	La notación IE (Finkelstein, 1989) es sencilla y fácil de leer, y resulta idónea de alto nivel. Su único inconveniente, que podría considerarse una ventaja

	entidad. Se presupone que dichos atributos se modelarán con otro diagrama de documentación de apoyo.
Pregonero	La notación de Barker es una de las más populares, es compatible con la mayoría de modelos de datos. Su enfoque para la subtipificación puede volverse confuso.
IDEF1X	Esta notación es excesivamente compleja. Originalmente se concibió para ser aplicada erróneamente al modelado lógico. Si bien es popular en algunas agencias, como el Departamento de Defensa (DoD), prácticamente ha sido abandonada.
UML	Esta no es una notación oficial de modelado de datos (todavía). Si bien hay una notación de datos para UML , ninguna es completa y, lo que es más importante, a finales de 2005, el Object Management Group (OMG) anunció una solicitud de patentes.

3. Cómo modelar datos

Es fundamental que un desarrollador de aplicaciones comprenda los fundamentos del modelado de datos para que no solo pueda interpretar los modelos de datos, sino también colaborar eficazmente con los ingenieros de datos ágiles responsables de los aspectos relacionados con los datos de su iniciativa. El objetivo de leer esta sección no es aprender a ser un modelador de datos, sino simplemente comprender lo que implica.

Las siguientes tareas se realizan de forma iterativa:

- [Identificar tipos de entidades](#)
- [Identificar atributos](#)
- [Aplicar convenciones de nomenclatura](#)
- [Identificar relaciones](#)
- [Aplicar patrones de modelos de datos](#)
- [Asignar claves](#)

- [Normalizar para reducir la redundancia de datos.](#)
- [Desnormalizar para mejorar el rendimiento](#)

Entre los libros prácticos más recomendables sobre modelado de datos se encuentran [*Data & Databases*](#) y [*Data Modeling for Information Professionals*](#), de Joe Celko, ya que ambos se centran en aspectos prácticos del modelado de datos. [*Data Modeling Handbook*](#) y [*Data Model Patterns*](#) son excelentes recursos una vez que se dominan los fundamentos. [*An Introduction to Database Systems*](#) es un buen tratado académico para quienes deseen especializarse en datos.

3.1 Identificar tipos de entidades

Un tipo de entidad, también llamado simplemente entidad (término no del todo preciso, pero muy común en la práctica), es conceptualmente similar al concepto de clase en la orientación a objetos: un tipo de entidad representa una colección de objetos similares. Un tipo de entidad podría representar una colección de personas, lugares, cosas, eventos o conceptos. Ejemplos de entidades en un sistema de entrada de pedidos serían *Cliente*, *Dirección*, *Pedido*, *Artículo* e *Impuesto*. Si estuviera modelando clases, esperaría encontrar clases con los mismos nombres. Sin embargo, la diferencia entre una clase y un tipo de entidad es que las clases tienen tanto datos como comportamiento, mientras que los tipos de entidad solo tienen datos.

Idealmente, una entidad debería ser [normal](#), el equivalente a la cohesión en el mundo del modelado de datos. Una entidad normal representa un solo concepto, al igual que una clase cohesiva modela un solo concepto. Por ejemplo, cliente y pedido son claramente dos conceptos diferentes; por lo tanto, tiene sentido modelarlos como entidades separadas.

3.2 Identificar atributos

Cada tipo de entidad tendrá uno o más atributos de datos. Por ejemplo, en [la Figura 1](#) se observa que la entidad *Cliente* tiene atributos como *Nombre* y *Apellido*, y en la [Figura 2](#) se muestra que la tabla *TCUSTOMER* tiene las columnas de datos correspondientes *CUST_FIRST_NAME* y *CUST_SURNAME* (una columna representa un atributo de datos en una base de datos relacional).

Los atributos también deben ser coherentes desde el punto de vista de su dominio, algo que a menudo es una decisión subjetiva. – En la Figura 1 decidimos que queríamos modelar el hecho de que las personas tenían nombre y apellido en lugar de solo un nombre (por ejemplo, "Scott" y "Ambler" frente a "Scott Ambler") mientras que no distinguimos entre las secciones de un código postal estadounidense (por ejemplo, 90210-1234-5678). Lograr el nivel de detalle correcto puede tener un impacto significativo en sus esfuerzos de desarrollo y mantenimiento. Refactorizar una sola columna de datos en varias columnas puede ser difícil, la refactorización de bases de datos se describe en detalle en [Refactorización de bases de datos](#) , aunque especificar en exceso un atributo (por ejemplo, tener tres atributos para el código postal cuando solo se necesitaba uno) puede resultar en una construcción excesiva de su sistema y, por lo tanto, incurrir en mayores costos de desarrollo y mantenimiento de los realmente necesarios.

3.3 Aplicar convenciones de nomenclatura de datos

Su organización debería contar con estándares y directrices aplicables al modelado de datos, información que debería poder obtener de sus administradores de bases de datos (si no existen convenciones, debería solicitar su implementación). Estas directrices deben incluir convenciones de nomenclatura tanto para el modelado lógico como para el físico. Las convenciones de nomenclatura lógica deben centrarse en la legibilidad humana, mientras que las convenciones de nomenclatura física reflejarán consideraciones técnicas. En las figuras 1 y 2 se observa claramente que se aplicaron diferentes convenciones de nomenclatura .

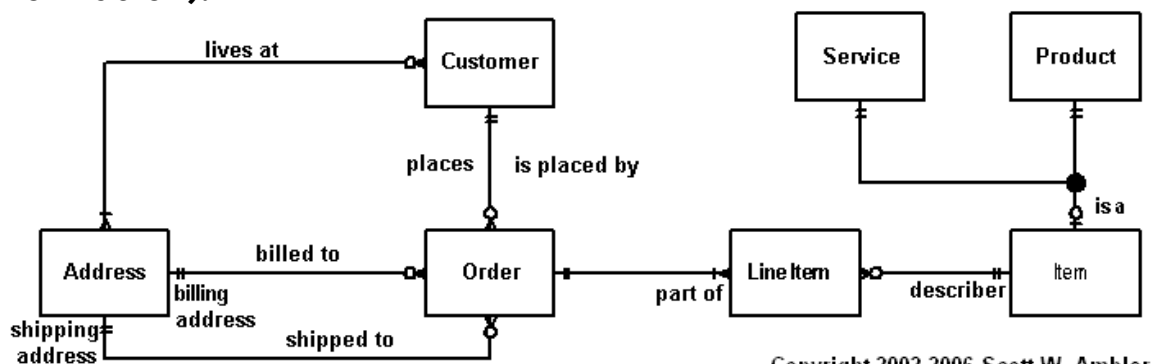
Como viste en [la Introducción al Modelado Ágil](#) , el Modelado Ágil incluye la práctica [de Aplicar Estándares de Modelado](#) . La idea básica es que los desarrolladores deben acordar y seguir un conjunto común de estándares de modelado en una [iniciativa](#) de software . Así como es valioso seguir convenciones de codificación comunes (un código limpio que sigue las pautas de codificación elegidas es más fácil de entender y evolucionar que un código que no las sigue), también lo es seguir convenciones de modelado comunes.

3.4 Identificar relaciones

En el mundo real, las entidades tienen relaciones entre sí. Por ejemplo, los clientes hacen pedidos, los clientes viven en determinadas direcciones y los artículos forman parte de los pedidos. Los términos «hacer», «vivir en» y «formar parte de» definen las relaciones entre entidades. Estas relaciones son conceptualmente idénticas a las relaciones (asociaciones) entre objetos.

La Figura 5 muestra un LDM parcial para un sistema de pedidos en línea. Lo primero que se observa son los distintos estilos aplicados a los nombres y roles de las relaciones: cada relación requiere un enfoque diferente. Por ejemplo, la relación entre *Cliente* y *Pedido* tiene dos nombres, " *places*" y " *is place by*", mientras que la relación entre *Cliente* y *Dirección* tiene uno. En este ejemplo, tener un segundo nombre en la relación, con la idea de especificar cómo se interpreta en cada dirección, resulta redundante; es mejor usar una redacción clara para un único nombre de relación, lo que reduce la complejidad del diagrama. Del mismo modo, a menudo se observa que especificar los roles que desempeña una entidad en una relación elimina la necesidad de darle un nombre (aunque algunas herramientas CASE pueden obligar inadvertidamente a hacerlo). Por ejemplo, los roles de *dirección de facturación* y etiqueta " *facturado a*" son claramente redundantes; solo se necesita uno. Por ejemplo, el rol que tiene *parte de ese Artículo* en su relación con *Pedido* es suficientemente obvio sin necesidad de un nombre de relación.

Figura 5. Modelo lógico de datos (notación de ingeniería de la información).



También es necesario identificar la cardinalidad y la opcionalidad de una relación ([UML](#) combina los conceptos de opcionalidad y cardinalidad en el concepto único de multiplicidad). La cardinalidad representa el concepto de "cuántos", mientras que la opcionalidad representa el concepto de "si es necesario tener algo". Por ejemplo, no basta con saber que los clientes hacen pedidos. ¿Cuántos pedidos puede hacer un cliente? ¿Ninguno, uno o varios? Además, las relaciones son bidireccionales: los clientes no solo hacen pedidos, sino que también los hacen. Esto plantea preguntas como: ¿cuántos clientes pueden estar inscritos en un pedido determinado y es posible tener un pedido sin ningún cliente involucrado? [La Figura 5](#) muestra que los clientes hacen cero o más pedidos y que cualquier pedido determinado lo hace un solo cliente. También muestra que un cliente vive en una o más direcciones y que cualquier dirección determinada tiene cero o más clientes que viven en ella.

Aunque UML distingue entre diferentes tipos de relaciones (asociaciones, herencia, agregación, composición y dependencia), los modeladores de datos a menudo no se preocupan tanto por este tema como los modeladores de objetos. El subtipado, una aplicación de la herencia, se encuentra frecuentemente en los modelos de datos; un ejemplo de ello es la *relación* entre *Item* y sus dos subentidades, *Service* y *Product*. La agregación y la composición son mucho menos comunes y, por lo general, deben inferirse del modelo de datos, como se observa en la *parte del* rol que desempeña *Line Item con Order*. Las dependencias UML suelen ser una construcción de software y, por lo tanto, no aparecerían en un modelo de datos, a menos que se tratara de un modelo físico muy detallado que mostrara cómo las vistas, los disparadores o los procedimientos almacenados dependen de otros aspectos del esquema de la base de datos.

3.5 Aplicar patrones de modelos de datos

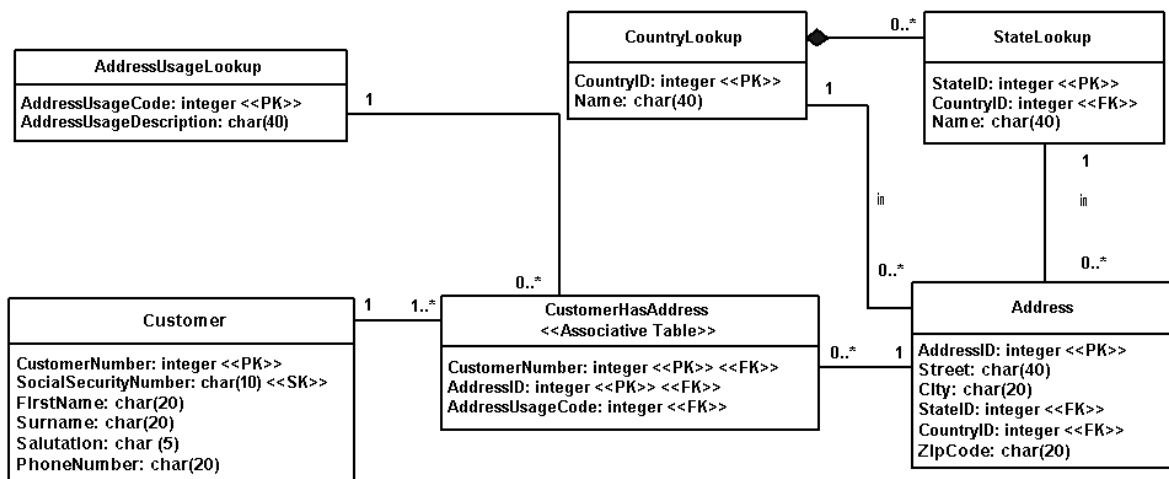
Algunos modeladores de datos aplicarán patrones comunes de modelos de datos, como se describe en el libro de David Hay, [Data Model Patterns](#). Es la mejor referencia sobre el tema, al igual que los desarrolladores orientados a objetos aplican patrones de análisis y [patrones de diseño](#). Los patrones de modelos de datos son

conceptualmente más cercanos a los patrones de análisis porque describen soluciones a problemas comunes del dominio. El libro de Hay es una excelente referencia para cualquiera que participe en el modelado a nivel de análisis, incluso cuando se adopta un enfoque orientado a objetos en lugar de uno orientado a datos, ya que sus patrones modelan estructuras empresariales de una amplia variedad de dominios.

3.6 Asignar teclas

Existen dos estrategias fundamentales para asignar claves a las tablas. En primer lugar, se puede asignar una clave natural, que consiste en uno o más atributos de datos existentes que son únicos para el concepto de negocio. En la tabla *Cliente* de la Figura 6, había dos claves candidatas: *CustomerNumber* y *SocialSecurityNumber*. En segundo lugar, se puede introducir una nueva columna, denominada clave subrogada, que es una clave sin significado para el negocio. Un ejemplo de ello es la columna *AddressID* de la tabla *Dirección* en la Figura 6. Las direcciones no tienen una clave natural "sencilla", ya que se necesitarían todas las columnas de la tabla *Dirección* para formar una clave propia (quizás bastaría con la combinación de *Calle* y *Código Postal*, dependiendo del dominio del problema); por lo tanto, introducir una clave subrogada es una opción mucho mejor en este caso.

Figura 6. Cliente y dirección revisados ([notación UML](#)).



Analicemos [la Figura 6](#) con más detalle. La Figura 6 presenta un diseño alternativo al de la [Figura 2](#), con una convención de nomenclatura diferente y un modelo más extenso. En la Figura 6, la tabla *Cliente* tiene la columna *NúmeroCliente* como clave primaria y *NúmeroSeguridadSocial* como clave alternativa. Esto indica que la forma preferida de acceder a la información del cliente es mediante el número de cliente, aunque el software puede obtener la misma información si dispone del número de seguridad social. La tabla *ClienteTieneDirección* tiene una clave primaria compuesta, formada por *NúmeroCliente* e *ID de Dirección*. Una clave externa es uno o más atributos de un tipo de entidad que representan una clave, primaria o secundaria, en otro tipo de entidad. Las claves externas se utilizan para mantener relaciones entre filas. Por ejemplo, la relación entre las filas de la tabla *ClienteTieneDirección* y la tabla *Cliente* se mantiene mediante la columna *NúmeroCliente* de la tabla *ClienteTieneDirección*. Lo interesante de la columna *CustomerNumber* es que forma parte de la clave primaria de *CustomerHasAddress* y también es clave foránea de la tabla *Customer*. De manera similar, la columna *AddressID* forma parte de la clave primaria de *CustomerHasAddress* y también es clave foránea de la tabla *Address* para mantener la relación con las filas de *Address*. Aunque el debate entre claves naturales y claves sustitutas es uno de los temas más controvertidos en el ámbito de los datos, lo cierto es que ninguna estrategia es perfecta y, en la práctica (como se muestra en [la Figura 6](#)), a veces conviene usar claves naturales y otras veces claves sustitutas. En [«Elegir una clave primaria: ¿natural o sustituta?»](#) describo en detalle las cuestiones relevantes.

3.7 Normalizar para reducir la redundancia de datos

La [normalización de datos](#) es un proceso en el que se organizan los atributos de datos dentro de un modelo de datos para aumentar la cohesión de los tipos de entidades. En otras palabras, el objetivo de la normalización de datos es reducir e incluso eliminar la redundancia de datos, una consideración importante para los desarrolladores de aplicaciones, ya que resulta extremadamente difícil almacenar objetos

en una base de datos relacional que mantiene la misma información en varios lugares.

La [Tabla 2](#) resume las tres reglas de normalización más comunes que describen cómo organizar los tipos de entidad en una serie de niveles crecientes de normalización. Los niveles superiores de normalización de datos (Date 2000) quedan fuera del alcance de este libro. En cuanto a la terminología, se considera que un esquema de datos está en el nivel de normalización de su tipo de entidad menos normalizado. Por ejemplo, si todos sus tipos de entidad están en la segunda forma normal (2FN) o superior, decimos que su esquema de datos está en 2FN.

Tabla 2. Reglas de normalización de datos.

Nivel	Regla
Primera forma normal (1FN)	Un tipo de entidad está en 1NF cuando no contiene grupos
Segunda forma normal (2FN)	Un tipo de entidad está en 2NF cuando está en 1NF y cuando no contiene dependencias transitivas de su clave primaria.
Tercera forma normal (3FN)	Un tipo de entidad está en 3NF cuando está en 2NF y cuando no contiene dependencias transitivas de su clave primaria.

La [figura 7](#) muestra un esquema de base de datos en ONF, mientras que la [figura 8](#) muestra un esquema normalizado en 3NF. Para más detalles, consulte el ensayo [«Introducción a la normalización de datos»](#).

¿Por qué la normalización de datos? La ventaja de tener un esquema de datos altamente normalizado es que la información se almacena en un solo lugar, lo que reduce la posibilidad de datos inconsistentes.

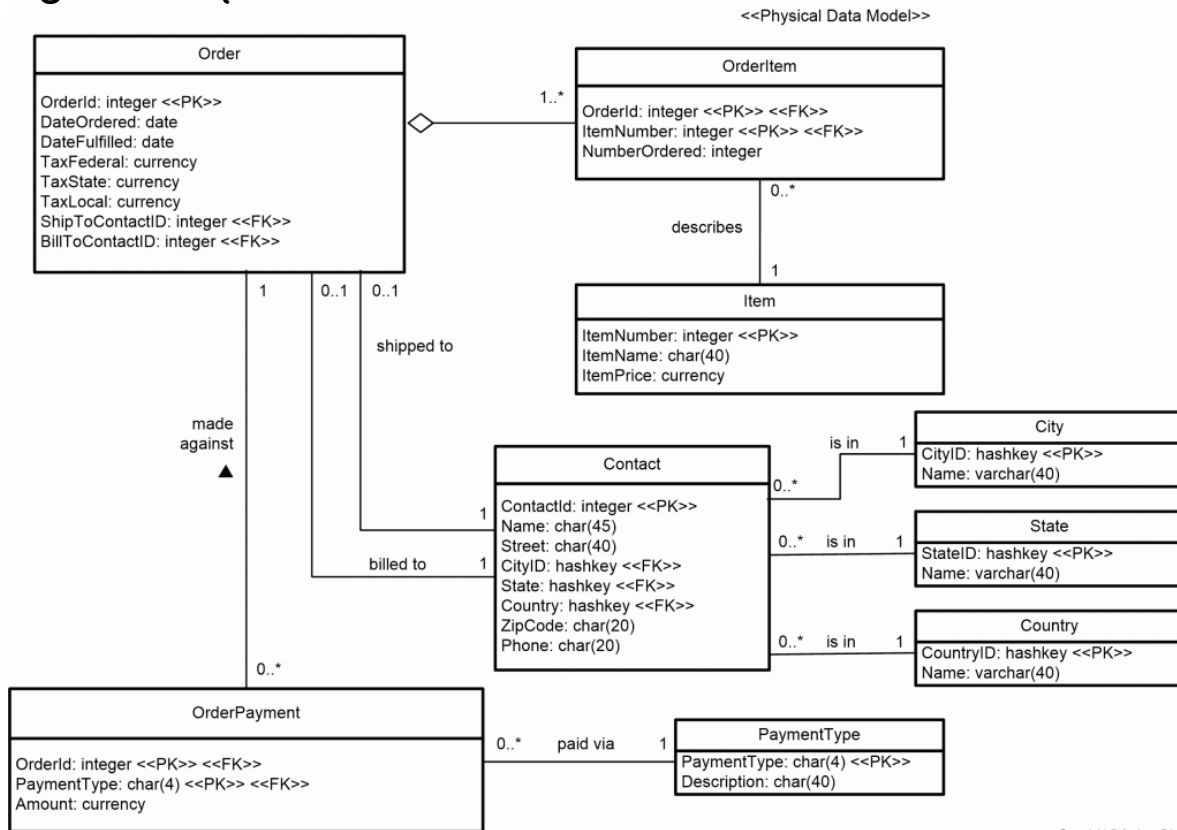
Además, los esquemas de datos altamente normalizados en general están conceptualmente más cerca de los esquemas orientados a objetos, ya que los objetivos de la orientación a objetos de promover una alta cohesión y un acoplamiento flexible entre clases dan como resultado soluciones similares (al menos desde el punto de vista de los datos). Esto generalmente facilita el [mapeo de los objetos al esquema de datos](#). Desafortunadamente, la normalización suele tener un costo en el rendimiento. Con el esquema de datos de [la Figura 7](#), todos los datos de un solo pedido se almacenan en una sola fila (suponiendo pedidos de hasta nueve artículos), lo que facilita mucho el acceso. Con el esquema de datos de [la Figura 7](#), se podría determinar rápidamente el monto total de un pedido leyendo la única fila de la tabla *OrderONF*. Para hacer lo mismo con el esquema de datos de [la Figura 8](#), sería necesario leer los datos de una fila de la tabla *Order*, los datos de todas las filas de la tabla *OrderItem* para ese pedido y los datos de las filas correspondientes de la tabla *Item* para cada artículo del pedido. Para esta consulta, es muy probable que el esquema de datos de [la Figura 7](#) proporcione un mejor rendimiento.

Figura 7. Esquema de datos inicial para el pedido ([notación UML](#)).

Order0NF

OrderId: integer <<PK>>
DateOrdered: date
DateFulfilled: date
Payment1Amount: currency
Payment1Type: char(4)
Payment1Description: char(40)
Payment2Amount: currency
Payment2Type: char(4)
Payment2Description: char(40)
TaxFederal: currency
TaxState: currency
TaxLocal: currency
SubtotalBeforeTax: currency
ShipToName: char(45)
ShipToStreet: char(40)
ShipToCity: char(20)
ShipToState: char(20)
ShipToCountry: char(20)
ShipToZipCode: char(20)
ShipToPhone: char(20)
BillToName: char(45)
BillToStreet: char(40)
BillToCity: char(20)
BillToState: char(20)
BillToCountry: char(20)
BillToZipCode: char(20)
BillToPhone: char(20)
ItemName1: char(40)
ItemNumber1: integer
NumberOrdered1: integer
InitialItemPrice1: currency
TotalPriceExtended1: currency
ItemName2: char(40)
ItemNumber2: integer
NumberOrdered2: integer
InitialItemPrice2: currency
TotalPriceExtended2: currency
...
ItemName9: char(40)
ItemNumber9: integer
NumberOrdered9: integer
InitialItemPrice9: currency
TotalPriceExtended9: currency

Figura 8. Esquema normalizado en 3NF (**Notación UML**).



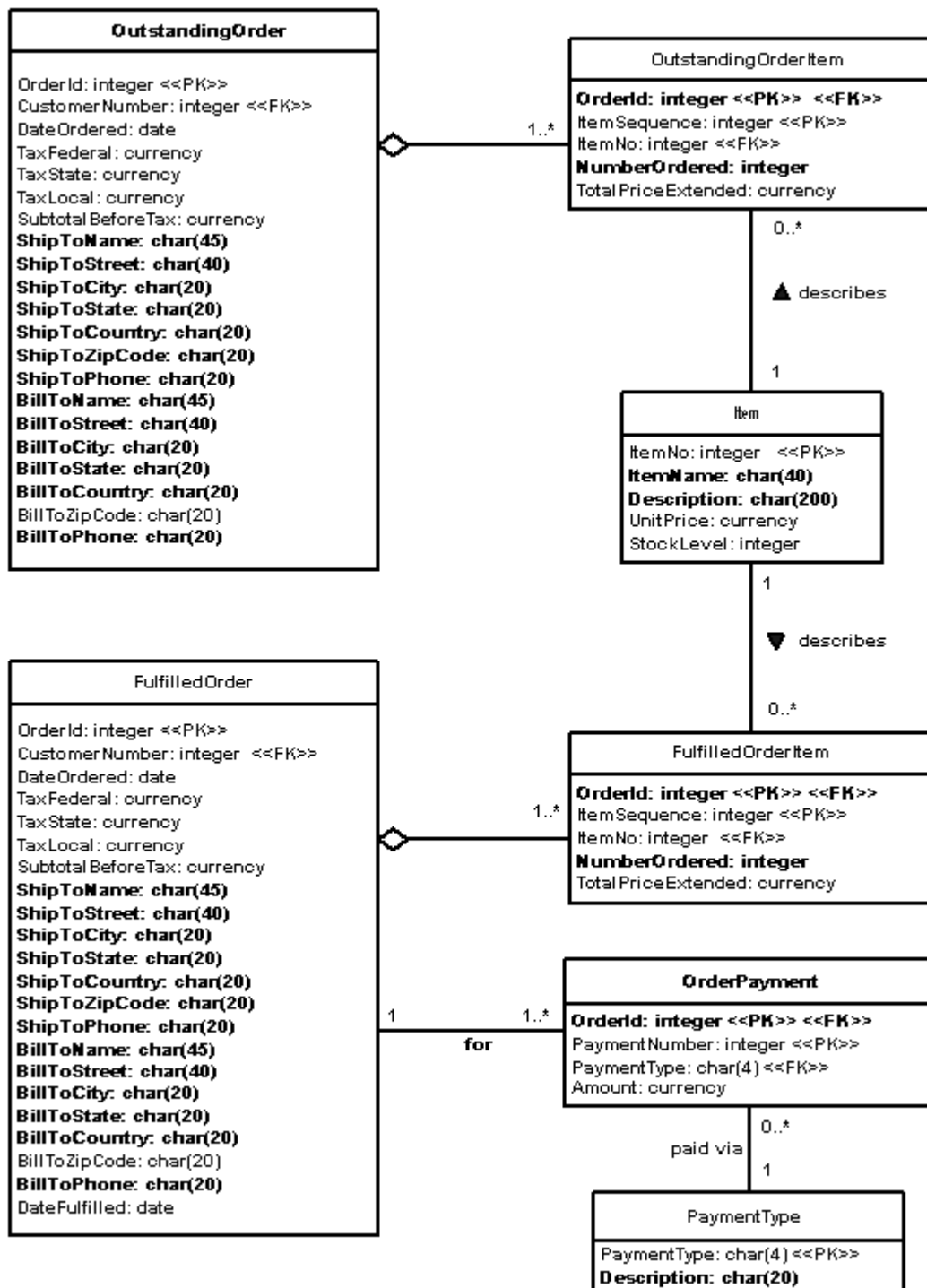
En el modelado de clases, existe un concepto similar llamado **normalización de clases** , aunque eso queda fuera del alcance de este artículo.

3.8 Desnormalizar para mejorar el rendimiento

Los esquemas de datos normalizados, al implementarse en producción, suelen presentar problemas de rendimiento. Esto es lógico, ya que las reglas de normalización de datos se centran en reducir la redundancia de datos, no en mejorar el rendimiento del acceso a los mismos. Una parte importante del modelado de datos consiste en desnormalizar partes del esquema para optimizar los tiempos de acceso a la base de datos. Por ejemplo, el modelo de datos de **la Figura 9** no se parece en nada al esquema normalizado de **la Figura 8**. Para comprender las diferencias entre ambos esquemas, es necesario considerar las necesidades de rendimiento de la aplicación. El objetivo principal de

este sistema es procesar los nuevos pedidos de los clientes en línea lo más rápido posible. Para ello, los clientes deben poder buscar artículos y añadirlos a su pedido con rapidez, eliminarlos si es necesario y, finalmente, obtener el total y el registro del pedido con agilidad. El objetivo secundario del sistema es procesar, enviar y facturar los pedidos posteriormente.

Figura 9. Esquema de datos de orden desnormalizado ([notación UML](#)).



Para desnormalizar el esquema de datos se tomaron las siguientes decisiones:

1. La tabla *de artículos* se dejó sin modificar para facilitar la búsqueda rápida de información sobre los mismos.
2. Para facilitar la adición y eliminación de artículos a un pedido, se mantuvo el concepto de la tabla *OrderItem*,

*aunque dividida en dos para gestionar los pedidos pendientes y los pedidos completados. Los nuevos artículos se pueden insertar o eliminar fácilmente de la tabla *OutstandingOrderItem* según sea necesario.*

3. Las tablas *Order* y *OrderItem* se reorganizaron en pares para gestionar los pedidos pendientes y los pedidos completados, respectivamente. La información básica del pedido se almacena primero en las tablas *OutstandingOrder* y *OutstandingOrderItem* , y luego, una vez que el pedido se ha enviado y pagado, los datos se eliminan de esas tablas y se copian en las tablas *FulfilledOrder* y *FulfilledOrderItem* , respectivamente. El tiempo de acceso a los datos de las dos tablas para los pedidos pendientes se reduce porque solo se almacenan allí los pedidos activos. En promedio, un pedido puede permanecer pendiente durante un par de días, mientras que, por motivos de información financiera, puede almacenarse en las tablas de pedidos completados durante varios años hasta su archivo. Este esquema conlleva una penalización en el rendimiento debido a la necesidad de eliminar los pedidos pendientes y luego volver a guardarlos como pedidos completados, lo que obviamente requeriría un procesamiento como una transacción.
4. La información de contacto de la(s) persona(s) a la(s) que se envía y factura el pedido también se desnormalizó y se volvió a incorporar a la tabla *de Pedidos* , lo que redujo el tiempo necesario para escribir un pedido en la base de datos, ya que ahora se realiza una sola escritura en lugar de dos o tres. Los tiempos de recuperación y eliminación de esos datos también mejoraron de forma similar.

Tenga en cuenta que si el diseño inicial de sus datos normalizados satisface las necesidades de rendimiento de su aplicación, entonces está bien tal como está. La desnormalización solo debe utilizarse cuando las pruebas de rendimiento demuestren que existe un problema con sus objetos y el análisis posterior revele que es necesario mejorar el tiempo de acceso a la base de datos. Como decía mi abuelo: si algo funciona, no lo toques.

5. Evolutivo/Modelado de datos ágil

El modelado de datos evolutivo es un método iterativo e incremental. El artículo « [Desarrollo evolutivo](#)» explora con mayor detalle el desarrollo de software evolutivo. El modelado de datos ágil es un método evolutivo que se lleva a cabo de forma colaborativa. El artículo « [Modelado de datos ágil: del modelado de dominio al modelado físico](#)» presenta un estudio de caso que muestra cómo aplicar un enfoque ágil al modelado de datos.

Aunque no lo parezca, el modelado de datos puede ser una de las tareas más desafiantes para un ingeniero de datos ágil en un equipo de desarrollo de software ágil. Tu enfoque del modelado de datos suele ser el centro de cualquier controversia entre los desarrolladores de software ágil y los profesionales de datos tradicionales de tu organización. Los desarrolladores de software ágil tienden a adoptar un enfoque evolutivo, donde el modelado de datos es solo una de las muchas actividades, mientras que los profesionales de datos tradicionales suelen inclinarse por un enfoque [de diseño integral \(BDUF, por sus siglas en inglés\)](#), donde los modelos de datos son los artefactos principales, si no los principales. Este problema surge de una combinación de la [incompatibilidad cultural](#), la necesidad errónea de imponer la "única verdad" y las maniobras políticas habituales dentro de la organización. Como resultado, los ingenieros de datos ágiles a menudo descubren que navegar por las aguas políticas es una parte importante de sus esfuerzos de modelado de datos.

6. Cómo Mejora tus habilidades para modelar datos.

¿Cómo mejorar tus habilidades de modelado de datos? Practica, practica y practica. Siempre que tengas la oportunidad, trabaja en estrecha colaboración con ingenieros de datos ágiles, ofrécete como voluntario para [modelar datos con ellos](#) y hazles preguntas a medida que avanza el trabajo. Los ingenieros de datos ágiles seguirán la [práctica de modelado](#) colaborativo de AM, por lo que agradecerán tanto la ayuda como las preguntas; una de las mejores maneras de aprender realmente es que alguien te pregunte "¿por qué lo haces de esa manera?". Deberías poder aprender habilidades de modelado de datos físicos de los ingenieros de datos ágiles, y a menudo también habilidades de modelado de datos lógicos.

De igual manera, deberías aprovechar la oportunidad de trabajar con los arquitectos empresariales de tu organización. Como viste en [Arquitectura Empresarial Ágil](#), deberían participar activamente en tu equipo, asesorándolos en arquitectura empresarial (si la hay), brindándote orientación sobre modelado y habilidades arquitectónicas, y apoyando los esfuerzos de modelado y desarrollo de tu equipo. Una vez más, ofréctete como voluntario para trabajar con ellos y no dudes en hacer preguntas. Los arquitectos empresariales podrán enseñarte habilidades de modelado de datos conceptuales y lógicos, además de inculcarte una comprensión profunda de los desafíos empresariales. También es necesario leer un poco. Si bien este artículo es un buen punto de partida, es solo una breve introducción. Lo mejor es preguntarles a los ingenieros de datos ágiles con los que trabajas qué lecturas recomiendan.

Mi último consejo es que es fundamental que los desarrolladores de aplicaciones comprendan y valoren los fundamentos del modelado de datos. Esta es una habilidad valiosa que se ha mantenido desde la década de 1970. Además, proporciona un marco común para trabajar con ingenieros de datos ágiles e incluso puede ser la habilidad inicial que les permita dar el salto profesional y convertirse en ingenieros de datos ágiles de pleno derecho.